That concludes our implementation of the REST API that exposes the Catalog Service's book-management functionality. Next I'll discuss a few aspects of how we can evolve APIs to adapt to new requirements.

3.3.4 Evolving APIs for future requirements

In a distributed system, we need a plan to evolve APIs so we don't break the functionality of other applications. This is a challenging task because we want independent applications, but they probably exist to provide services to other applications, so we are somewhat limited in the number of changes we can make independently of the clients.

The best approach is to make backward-compatible changes to the API. For example, we can add an optional field to the `Book` object without affecting the clients of the Catalog Service application.

Sometimes, breaking changes are necessary. In this situation, you can use *API versioning*. For example, should you decide to make a breaking change to the REST API for the Catalog Service application, you might introduce a versioning system for the endpoints. The version might be part of the endpoint itself, like `/v2/books`. Or it might be specified as an HTTP header. This system helps prevent existing clients from breaking, but they will have to update their interface to match the new API version sooner or later, meaning that coordination is needed.

A different approach focuses on making the REST API client as resilient to API changes as possible. The solution is to use the *hypermedia* aspect of the REST architecture, as described by Dr. Roy Fielding in his doctorate dissertation, "Architectural Styles and the Design of Network-based Software Architectures" (www.ics.uci.edu/~fielding/pubs/dissertation/top.htm). REST APIs can